

Linguagens de Programação

Índice de Aulas

1. [10/08 - Introdução - A grande diversidade](#)
2. [12/08 - Sintaxe](#)
3. [17/08 - Mais sobre árvores de sintaxe](#)
4. [19/08 - Sistemas de Computação](#)
5. [24/08 - Introdução a ML](#)
6. [26/08 - Casamento de padrões em ML](#)
7. [31/08 - Tipos de dados](#)
8. [02/09 - Polimorfismo](#)
9. [12/09 - O cálculo lambda](#)
(Sábado)
10. [14/09 - Funções de alta ordem](#)
11. [16/09 - Escopo de variáveis](#)
12. [21/09 - Tipos algébricos](#)
23/09 - Aula de revisão
28/09 - Primeira avaliação
13. [30/09 - Registros de ativação](#)
14. [05/10 - Introdução à linguagem Python](#)
15. [07/10 - Gerenciamento de memória](#)
16. [19/10 - Tipos abstratos de dados](#)
17. [21/10 - Programação orientada a objetos](#)
18. [26/10 - Tratamento de erros](#)
19. [28/10 - Passagem de parâmetros](#)
20. [04/11 - Introdução à linguagem Prolog](#)
21. [09/11 - Modelos de Custos](#)
22. [11/11 - Predicados numéricos em Prolog](#)
23. [16/11 - Semântica Formal](#)
24. [18/11 - História das linguagens de programação](#)
23/11 - Aula de revisão
25/11 - Segunda avaliação

30/11 - **Prova Suplementar**
02/12 - **Exame Especial**

Introdução

Conceitos que devem ser entendidos.

- Existem linguagens de programação imperativas, funcionais e lógicas. A fronteira entre estes paradigmas não é bem definida.
- Diferentes paradigmas facilitam a implementação de diferentes tipos de aplicações.

Questões para discutir

- Por que existem linguagens de programação?
- Existe uma linguagem de programação mais "poderosa"?
- Por que existem **tantas** linguagens de programação?
- Por que algumas linguagens de programação são mais populares que outras?

- Por que algumas linguagens de programação são mais eficientes que outras?
- O que é uma boa linguagem de programação?
- Por que é importante aprender sobre linguagens de programação?
- Considere a sua linguagem de programação favorita. Há algo que você adicionaria ou mudaria nesta linguagem?
- Pense em exemplos de aplicações que são mais facilmente implementadas em cada um dos três paradigmas principais de linguagens de programação.

Leitura

- [Introdução à Linguagens de Programação](#)
- [Página do Laboratório de Compiladores](#)
- [A pesquisa em compiladores na UFMG](#)
- Why you should learn other [languages](#).
- **Importante:** Capítulo 1 do livro [Introduction to Programming Languages](#).

Exercícios

1. Para ser entregue: [Lista 1](#). Lembre-se: respostas escritas à mão.
2. (P.O.F): Assista um filme em casa com pessoas de quem você goste.

[Notas e exemplos](#) usados em aula.

Sintaxe

Conceitos que devem ser entendidos.

- A sintaxe e estrutura léxica de uma linguagem de programação determinam a aparência de seus programas
- A semântica de uma linguagem de programação determina o significado de cada um de seus construtos.
- Como escrever uma gramática simples.

Questões para discutir

- Haveria algum outro jeito de descrever uma linguagem de programação?
- Qual o problema de representar os tokens dentro da gramática?

Leitura

- [Sintaxe de Linguagens de Programação](#) Seções 1.1, 1.2 e 1.3.
- Um pouco de motivação com o famoso triângulo de [Sierpinski](#).
- Uma linguagem que gera sistemas-L: [LinF](#).
- Exemplo de gramática. Neste caso, trata-se de [minijava](#), um subconjunto de Java usado em trabalhos práticos.
- **Importante:** Capítulo 2 do livro [Introduction to Programming Languages](#).

Exercícios

1. Para ser entregue: [Lista 2](#). Lembre-se: respostas escritas à mão.
2. (P.O.F): Abraça uma pessoa pela primeira vez.

[Notas e exemplos](#) usados em aula.

Mais sobre árvores de sintaxe.

Conceitos que devem ser entendidos.

- Precedência entre operadores
- Associatividade
- Gramáticas ambíguas permitem a construção de duas diferentes árvores de sintaxe para o mesmo texto.
- Árvores de sintaxe abstrata

Questões para discutir

- Quantos níveis de precedência possui a sua linguagem favorita?
- O que significa este trecho de código C: "a = b < c ? * p + b * c : 1 << d ()"?
- Qual é a história de gramáticas em linguagens de programação?

Leitura

- [Sintaxe de Linguagens de Programação](#) Seções 1.4.
- [Gramáticas em Prolog](#) Páginas 46 (Prolog Grammar Rules) até 50.
- A [história](#) de Métodos de Parsing.
- [Tabela](#) de precedência em Java.
- [Tabela](#) de precedência em C.
- **Importante:** Capítulo 3 do livro [Introduction to Programming Languages](#).

Exercícios

1. Para ser entregue: [Lista 3](#). Lembre-se: respostas escritas à mão.
2. (P.O.F.): Encontre uma constelação no céu.

[Notas e exemplos](#) usados em aula.

Sistemas de Computação

Conceitos que devem ser entendidos.

- Pre-processadores. Ex: gcc -E
- Compiladores. Ex: cc1 (experimente gcc -S para ver o assembly)
- Montadores. Ex: as
- Ligadores. Ex: ld
- Carregadores. Ex: ./a.out
- Interpretadores. Ex: bash
- Máquinas virtuais. Ex: JavaVM
- Depuradores. Ex: gdb
- Just in time Compilers. Ex: Mozilla lendo Javascript.
- Tempo de ligação:
 - Durante a definição da linguagem
 - Durante a implementação da linguagem
 - Durante a compilação
 - Durante a ligação (link time)
 - Durante o carregamento (load time)
 - Durante a execução (run time).

Questões para discutir

- O que acontece quando você digita `gcc main.c ; ./a.out` na linha de comando e aperta `enter`?
- O que acontece quando você digita `javac Main.java ; java Main` na linha de comando e aperta `enter`?
- O que acontece quando seu navegador executa uma applet Java?
- Como funciona `gdb`?
- Quais as vantagens de um compilador sobre um interpretador? E vice-versa?

Leitura

- [Compiladores](#)
- [Debugadores](#)
- Um [tutorial](#) sobre `gdb`.
- [Interpretadores](#)
- [Máquinas virtuais](#)
- [Compilador just-in-time](#).
- **Importante:** Capítulo 4 do livro [Introduction to Programming Languages](#).

Exercícios

1. Para ser entregue: [Lista 4](#). Lembre-se: respostas escritas à mão. (Para o exercício 2, veja os [slides](#)) do livro.
2. (P.O.F.): Dê uma flor para uma pessoa que você encontre na rua, e que nunca tenha visto antes.

[Notas e exemplos](#) usados em aula.

Introdução à ML

Conceitos que devem ser entendidos.

- Máquina de Turing versus Cálculo Lambda.
- ML é uma linguagem funcional.
- O conceito de referência não é definido em ML básica.
- Tuplas e listas são tipos de dados em ML.
- Construtores de tipos, por exemplo `'*`, `'->` e `list`.
- Recursividade de Cauda

Questões para discutir

- Quais as semelhanças entre ML e Java?
- E quais as diferenças entre essas linguagens?
- Quais são as vantagens de se usar uma linguagem sem referências para dados mutáveis?

Leitura

- [SML](#)
- [Tutorial](#) sobre SML (contribuição do Wladston).

Exercícios

1. Para ser entregue: [Lista 5](#). Lembre-se: respostas escritas à mão.

2. (P.O.F.): Leia um texto que explique um ponto de vista oposto a algo em que você acredita fortemente.

[Notas e exemplos](#) usados em aula.

Casamento de Padrões em ML

Conceitos que devem ser entendidos.

- Casamento de padrões são típicos em linguagens funcionais.

Questões para discutir

- Por que linguagens como C ou Java não possuem casamento de padrões?

Leitura

- [Casamento de padrões](#)

Exercícios

1. Para ser entregue: [Lista 6](#). Lembre-se: respostas escritas à mão.
Por favor, consulte os [slides](#) do livro para resolver estes exercícios.
2. (P.O.F.): Tome suco de uma fruta que você não conhece.

[Notas e exemplos](#) usados em aula.

Tipos de dados

Conceitos que devem ser entendidos.

- Um tipo de dado é um conjunto.
- Existem tipos de dados primitivos e compostos.
- Linguagens fortemente tipadas versus linguagens fracamente tipadas.
- Verificação de tipos dinâmica versus verificação estática.
- Equivalência de tipos estrutural versus equivalência nominal.

Questões para discutir

- Encontre exemplos de tipagem fraca na linguagem C.
- Compare verificação dinâmica versus estática.
- Compare equivalência estrutural versus equivalência nominal.
- Em equivalência estrutural (ex.: em Ocaml), $\text{int} * \text{int} * \text{int}$ é um subtipo de $\text{int} * \text{int}$. Explique este paradoxo.

Leitura

- [Tipos de dados](#)
- [What to know before debating about types](#)
- **Importante:** Capítulo 7 (Types) do livro [Introduction to Programming Languages](#).

Exercícios

1. Para ser entregue: [Lista 7](#). Lembre-se: respostas escritas à mão.
Por favor, consulte os [slides](#) do livro para resolver estes exercícios.
2. (P.O.F.): Dê algo seu que você use e goste para alguém que você acha que gostará mais ainda.

[Notas e exemplos](#) usados em aula.

Polimorfismo

Conceitos que devem ser entendidos.

- Uma função é polimórfica se ela tem pelo menos dois tipos diferentes.
- Polimorfismo ad-hoc (número finito de variações):
 - Overloading
 - Coerção
- Polimorfismo universal (número potencialmente infinito de variações):
 - Polimorfismo paramétrico
 - Polimorfismo de subtipagem

Questões para discutir

- Como polimorfismo facilita a programação?
- Compare polimorfismo paramétrico em Java e ML

Leitura

- [Polimorfismo de tipos](#)
- [Polimorfismo em linguagens orientadas a objetos.](#)

Exercícios

1. Para ser entregue: [Lista 8](#). Lembre-se: respostas escritas à mão.
2. (P.O.F.): Aprenda um poema de cor.

[Notas e exemplos](#) usados em aula.

O Cálculo Lambda

Conceitos que devem ser entendidos.

- O Cálculo Lambda é uma notação para descrever computações tão poderosa quanto o máquina de Turing.
- Variáveis livres.
- Funções Curricadas.
- Reduções Lambda:
 - Reduções alfa.
 - Reduções beta.
 - Reduções eta.
- Existem diferentes estratégias de redução.

Questões para discutir

- Por que Máquinas de Turing são utilizadas no ensino de fundamentos de computação, ao invés do Cálculo Lambda?

Leitura

- [O Cálculo Lambda](#). Capítulos 1-3.
- Representando [números](#) e valores booleanos usando lambdas.
- O cálculo Lambda para [desenvolvedores](#).
- [Slides para a aula](#).

Exercícios

1. Para ser entregue: [Lista 9](#). Lembre-se: respostas escritas à mão.
2. (P.O.F.): Tire uma foto do por do sol.

[Notas e exemplos](#) usados em aula.

Funções de Alta Ordem

Conceitos que devem ser entendidos.

- Uma função que não usa outra função como parâmetro ou valor de retorno possui ordem zero.
- Uma função que usa outra função como parâmetro ou valor de retorno possui ordem $n+1$, onde n é a mais alta ordem entre as funções usadas.
- `map`, `foldr` e `foldl` são funções de alta ordem.

Questões para discutir

- Como são funções de alta ordem em C?
- Como são funções de alta ordem em Java?
- Por que essas funções demoraram tanto para aparecer em Java?

Leitura

- [Funções de alta ordem](#)
- **Importante:** Capítulo 11 (High-Order Functions) do livro [Introduction to Programming Languages](#).

Exercícios

1. Para ser entregue: [Lista 10](#). Lembre-se: respostas escritas à mão.
2. (P.O.F.): Escreva uma carta para você mesmo no fim da graduação. Descreva como você se sente hoje, quais são seus desafios e o que você espera conseguir até lá.

[Notas e exemplos](#) usados em aula.

Escopo

Conceitos que devem ser entendidos.

- O escopo de uma definição é a parte do programa onde aquela definição é válida

- Linguagens de programação usam blocos e espaços de nomes para criar escopos
- Escopo estático versus escopo dinâmico
- Closures

Questões para discutir

- Qual é a vantagem de linguagens com escopo dinâmico.
- Quais os mecanismos usados por sua linguagem favorita para criar escopos?

Leitura

- [Structs e Signatures em SML](#)
- [Escopo](#)
- [Closures](#)

Exercícios

1. Para ser entregue: [Lista 11](#). Lembre-se: respostas escritas à mão.
2. (P.O.F.): Realize um P.O.F. tribal. Esse P.O.F. vale 0.5 pontos para cada membro do grupo que cumprir todas as [regras](#) (até dia 07/06/2026).

[Notas e exemplos](#) usados em aula.

Tipos Algébricos

Conceitos que devem ser entendidos.

- Em ML, novos tipos de dados são definidos com a palavra `datatype`
- `bool` e `list` são tipos algébricos
- Construtores de tipos.
 - Simples
 - Com valores-parâmetros
 - Recursivos

Questões para discutir

- Compare a implementação de árvores de dados em ML e em Java
- Quais os aspectos mais interessantes de ML?
- Por que linguagens funcionais são pouco populares?

Leitura

- [Tipos de dados algébricos.](#)
- [Datatypes em SML.](#)
- [Por que linguagens funcionais não são tão populares.](#)

Exercícios

1. Para ser entregue: [Lista 12](#). Lembre-se: respostas escritas à mão.
2. (P.O.F.): Ligue para a sua mãe para dizer-lhe que a ama.

[Notas e exemplos](#) usados em aula.

Registros de ativação

Conceitos que devem ser entendidos.

- As variáveis em um programa precisam ser armazenadas em algum lugar
- Registros de ativação (*activation record*)
 - Estático
 - Dinâmico
- Ponteiro de aninhamento (*nesting link*)
- Variáveis estáticas

Questões para discutir

- Como recursão de cauda é otimizada?
- O que existe em um registro de ativação?

Leitura

- [Pilha de Memória \(stack frame\)](#)
- [Registro de ativação](#)
- [Slides](#) sobre o assunto da aula.

Exercícios

1. Para ser entregue: [Lista 13](#). Lembre-se: respostas escritas à mão.
2. (P.O.F.): Faça um desenho de alguém de quem você goste. Meio ponto extra para quem postar em nossa lista de discussão um trabalho original, feito **neste** semestre (até dia 07/06/2026).

[Notas e exemplos](#) usados em aula.

Aula de Revisão.

Dicas para se dar bem na prova:

1. Entenda as notas e exemplos dados em aula.
2. Leia os [Slides](#) do livro.
3. Faça as listas de exercícios.
4. Faça todos os P.O.F's.

Abaixo estão alguns exemplos de provas dados em anos anteriores. Se quiserem uma dica, chamem seus colegas, amuquifem-se no reduto de algum deles, e dêem uma olhada em cada questão. Se ficarem cansados, parem tudo para escutar [Build Me Up Buttercup](#):

- [Prova 1.](#)
- [Prova 2.](#)
- [Prova 3.](#)
- [Prova 4.](#)
- [Prova 5.](#)
- [Prova 6.](#)
- [Prova 7.](#)
- [Prova 8.](#)
- [Prova 9.](#)
- [Prova 10.](#)
- [Prova 11.](#)

- [Prova 12.](#)
- [Prova 13.](#)
- [Prova 14.](#)
- [Prova 15.](#)
- [Prova 16.](#)
- [Prova 17.](#)
- [Prova 18.](#)
- [Prova 19.](#)
- [Prova 20.](#)
- [Prova 21.](#)
- [Prova 22.](#)
- [Prova 23.](#)
- [Prova 24.](#)
- [Prova 25.](#)
- [Prova 26.](#)
- [Prova 27.](#)
- [Prova 28.](#)
- [Prova 29.](#)

Primeira avaliação.

Duração: 90 minutos.

Valor: 30 Pontos.

Conteúdo: Toda a matéria até a aula de revisão

Modo: Sem consulta.

Introdução à Linguagem Python

Conceitos que devem ser entendidos.

- Linguagens Imperativas
 - Referências
 - Comandos e iterações
 - Efeitos colaterais (*Side Effects*)
- Linguagens dinâmicas
 - Tipagem dinâmica
 - Execução de código carregado dinamicamente
 - Alocação dinâmica de memória
- Máquinas virtuais, Interpretadores

Questões para discutir

- Qual o por quê dos princípios de programação que norteiam o projeto de Python?
Em que outras linguagens estes princípios são vistos?
- Quais as vantagens e desvantagens de máquinas virtuais?

Leitura

- [Site oficial de Python](#)
- [Python wiki](#)
- O [tutorial](#) definitivo sobre Python.

Exercícios

1. Para ser entregue: [Lista 14](#). Lembre-se: respostas escritas à mão.
2. (P.O.F.): Escute uma música de algum cantor ou cantora da África que você não conhecia. Procure saber sobre o país de onde vem a música.

[Notas e exemplos](#) usados em aula.

Gerenciamento de memória

Conceitos que devem ser entendidos.

- Memory Heap versus Memory Stack
 - Função
 - Forma de implementação
- O gerenciamento do Heap
 - Como encontrar novos blocos vagos: First-fit e Best-fit
 - Fragmentação e coalescing (fusão)
 - Coleta de Lixo
 - Marcação e varredura (mark and sweep)
 - Cópia e coleta
 - Contagem de referências (reference counting)
- Erros devido ao mau uso da memória
 - Vazamentos (memory leak)
 - Ponteiro cego (dangling pointer)

Questões para discutir

- Como programadores gerenciam memória em C/C++?
- Valgrind

Leitura

- Diversas técnicas de [Gerenciamento de memória](#)
- [Coleta de Lixo](#)
- [Valgrind](#)

Exercícios

1. Para ser entregue: [Lista 15](#). Lembre-se: respostas escritas à mão.
Esta classe será necessária:
 - [HeapManager.py](#)
2. (P.O.F.): Envie uma carta pelo correio para alguém de quem você goste.

[Notas e exemplos](#) usados em aula.

Tipos Abstratos de Dados

Conceitos que devem ser entendidos.

- Um tipo abstrato de dados é um tipo definido pelas operações e dados ao invés de ser definido como um conjunto de valores.
- Diferentes linguagens fornecem diferentes recursos para a implementação de Tipos Abstratos de Dados:
 - **C**: descrição de interfaces em *header files* e implementações em arquivos c.

- **SML**: *signatures* para a descrição de interfaces e *structs* para as implementações.
- **Python**: classes sem mecanismos de encapsulação.
- **Java**: *interfaces* para a descrição de interfaces e *classes* para as implementações. Encapsulação de propriedades.
- Algumas linguagens, como Python, provêem herança múltipla (mas isto traz problemas).

Questões para discutir

- Herança múltipla: bom ou ruim?
 - Como simular herança múltipla?
- Como simular polimorfismo paramétrico?

Leitura

- [Tipos abstratos de dados](#)
- [Herança Múltipla](#)
- [Mixins](#)

Exercícios

1. Para ser entregue: [Lista 16](#). Lembre-se: respostas escritas à mão. As classes nos arquivos [List.py](#) e [Worklist.py](#) serão necessárias.
2. (P.O.F.): Contribua para alguma página da wikipedia.

[Notas e exemplos](#) usados em aula.

Programação Orientada à Objetos

Conceitos que devem ser entendidos.

- Programação Orientada à Objetos não é exatamente um paradigma de linguagens de programação; é mais uma filosofia, um estilo de programação.
 - Visão antropomórfica de objetos: um objeto sabe como executar ações em si mesmo.
- Linguagens orientadas por objetos possuem algumas funcionalidades comuns:
 - Objetos.
 - Métodos.
 - Subtipagem.

Questões para discutir

- Por que a programação orientada por objetos é tão popular na indústria?

Leitura

- [Programação Orientada à Objetos](#)
- [POO em Python.](#)
- [POO em Java.](#)

Exercícios

1. Para ser entregue: [Lista 17](#). Lembre-se: respostas escritas à mão. Estas [classes](#) podem facilitar o seu trabalho.
2. (P.O.F.): Viagem com dois ou mais de seus colegas de aula.

[Notas e exemplos](#) usados em aula.

Tratamento de Erros

Conceitos que devem ser entendidos.

- Exceções são eventos anormais que podem ocorrer durante a execução de um programa.
- Mecanismos de tratamento de exceções em SML:
 - Podemos disparar exceções com `raise`.
 - Declaramos as exceções com `exception`.
 - Capturamos as exceções com `handle`.
- Mecanismos de tratamento de exceções em Python:
 - Blocos `try` e `except`.
 - Não é necessário estender uma classe em particular.
 - `finally`.
- Mecanismos de tratamento de exceções em Java:
 - Blocos `try` e `catch`.
 - Declarações `throws` e classes `throwable`.
 - `finally` e `finalize`.
- Em Java, exceções podem ser verificáveis (`checked`) ou não-verificáveis (`unchecked`).

Questões para discutir

- Qual é o papel de exceções em garantir a forte tipagem de linguagens tais como Python, Java e ML?
- Quais as vantagens e desvantagens entre exceções `checked` e `unchecked`?

Leitura

- Como exceções são implementadas em [C++](#).
- 10 boas [dicas](#) para o tratamento de exceções.
- [Tratamento de exceções em linguagens de programação](#).
- [Exceções em Python](#).
- Mais [exceções em Java](#).
- [Exceções checked e unchecked](#).

Exercícios

1. Para ser entregue: [Lista 18](#). Lembre-se: respostas escritas à mão. Talvez você queira usar as classes no arquivo [Emp.py](#).
2. (P.O.F.): Ligue para um amigo que você já não vê faz tempo.

[Notas e exemplos](#) usados em aula.

Passagem de parâmetros

Conceitos que devem ser entendidos.

- Parâmetros formais versus parâmetros reais.
- Correspondência posicional versus correspondência nominal.
 - Em correspondência posicional, parâmetros formais são casados com parâmetros reais de acordo com a posição na chamada da função. Exemplo: quase todas as linguagens de programação.
 - Em correspondência nominal, parâmetros são anexados a nomes, que os identificam durante a invocação da função. Exemplo: Ada.
- Tipos de passagem de parâmetros:
 - **Por valor** o parâmetro é como uma variável local no registro de ativação da função chamada, iniciada com o valor do parâmetro no momento em que a função foi chamada.
 - **Por resultado** o parâmetro é como uma variável local, porém não inicializada. Antes que a função retorne, um valor é copiado no endereço do parâmetro.
 - **Por valor e resultado** mistura passagem por valor e por resultados, isto é, o parâmetro é como uma variável local no registro de ativação da função chamada. Esta variável é inicializada com o valor do parâmetro real, antes da chamada da função. Antes que a função termine, um novo valor pode ser copiado naquela variável.
 - **Por referência** o endereço do parâmetro real é computado antes que a função seja chamada. Dentro do método, este endereço é usado como o endereço do parâmetro formal. Isto quer dizer que o parâmetro formal é um alias para o parâmetro real.
 - **Por expansão de macros** o corpo da macro é computado no contexto em que a função é chamada. Cada ocorrência do parâmetro formal é avaliado antes de ser usado.
 - **Por nome** cada parâmetro real é avaliado no contexto em que a função é chamada, cada vez que um parâmetro formal é usado.
 - **Por necessidade** cada parâmetro real é avaliado no contexto em que a função é chamada, mas somente na primeira vez que o correspondente parâmetro formal é usado. O resultado do processamento de um parâmetro é armazenado, para otimizar subsequentes usos.

Questões para discutir

- Quais as formas de passagem de parâmetros em Python, Java e C?
- Quais as vantagens e desvantagens da passagem de parâmetros por referências?

Leitura

- [Passagem de parâmetros](#)
- [Passagem de parâmetros em linguagens funcionais](#)
- **Importante:** Capítulo 20 (Parameter Passing) do livro [Introduction to Programming Languages](#).

Exercícios

1. Para ser entregue: [Lista 19](#). Lembre-se: respostas escritas à mão.
2. (P.O.F.): Escreva a lista das 100 melhores coisas do mundo. Para ajudar no exercício, aqui vai a minha [lista](#).

[Notas e exemplos](#) usados em aula.

Introdução à linguagens Prolog

Conceitos que devem ser entendidos.

- Prolog é uma linguagem de programação lógica.
- Programas escritos em prolog consistem em termos. Existem três tipos de termos: constantes (ou átomos), variáveis e termos compostos.
- Programas Prolog são executados via *queries*. Uma query pergunta ao interpretador se ele é capaz de provar alguma coisa.
- Regras são importantes termos. Uma regra define uma relação lógica, por exemplo, $A :- B, C$ significa que, se o interpretador consegue provar que B e C são verdade, então A é verdade também.
- Prolog é uma linguagem declarativa.
- Os nomes de átomos não têm significado particular para o interpretador Prolog. Eles são úteis somente para que humanos possam entender os programas.
- Assim como em ML, listas são importantes estruturas de dados em Prolog.
- Negação em Prolog é provada como a falha da query inversa.

Questões para discutir

- Prolog é uma boa linguagem para que tipos de aplicações?
- Quais as vantagens e desvantagens entre linguagens procedurais e declarativas?

Leitura

- [A linguagem Prolog](#)
- O manual [SWI Prolog](#). Vocês provavelmente usarão este ambiente de desenvolvimento.
- [Programação Lógica em Prolog](#).

Exercícios

1. Para ser entregue: [Lista 20](#). Lembre-se: respostas escritas à mão.
Para exs 1 - 6, use [estas](#) relações.
2. (P.O.F.): Vá soprar bolhas de sabão em uma praça pública.

[Notas e exemplos](#) usados em aula.

Unificação

Conceitos que devem ser entendidos.

- A essência de Prolog é um algoritmo chamado [unificação](#).
- **Importante:** nesta aula o aluno deve entender como representar uma query como uma árvore de busca.
- Prolog escolhe sempre o unificador mais geral (MGU - most general unifier) para unificar dois termos. Dizemos que U é mais geral que S se S é uma instância de U, mas o contrário não ocorre. Por exemplo, `parent(fred, Y)` é mais geral que `parent(fred, mary)`.
- Não se pode unificar termos de sintaxe diferente que incluem a mesma variável dos lados direito e esquerdo.
- Árvores de prova (clause-trees ou proof-trees).

Questões para discutir

- O que é mais complicado a respeito de Prolog, quando comparado com outras linguagens?

Leitura

- [Tutorial sobre unificação em Prolog](#) (Faça os exercícios).

Exercícios

1. Para ser entregue: [Lista 21](#). Lembre-se: respostas escritas à mão.
2. (P.O.F.): Faça uma lista de 50 coisas que você ainda gostaria de fazer durante a sua vida.

[Notas e exemplos](#) usados em aula.

Modelos de Custo

Conceitos que devem ser entendidos.

- Um modelo de custos descreve a complexidade das várias operações providas por uma linguagem de programação, em termos de tempo de execução e espaço.
 - Alguns modelos de custos são parte da especificação da linguagem, por exemplo, armazenamento de arranjos em C. Outros dependem da implementação da linguagem, como armazenamento de arranjos em Ada.
- Em Prolog e ML, a leitura do primeiro elemento de uma lista é $O(1)$. Reverter a lista, ou concatenar duas listas é $O(n)$.
- Recursão de cauda é uma otimização aplicada em funções recursivas, em que a chamada recursiva é a última operação realizada no corpo da função.
- Arranjos são armazenados em sequências de linhas, ou sequências de colunas, ou como vetores de vetores. O acesso sequencial de dados é sempre preferível.
- Buscas em Prolog apresentam um caráter exponencial. Às vezes o reordenamento de cláusulas melhora o programa.

Questões para discutir

- Até que ponto um programador deve conhecer a especificação de uma linguagem de programação?
- Existem operações que são inerentemente mais eficientes em linguagens que possuem arranjos?

Leitura

- [Chamada de cauda](#)
- [Array programming](#)
- [Recursão de cauda](#)
- **Importante:** Capítulo sobre *Cost Models* do livro [Introduction to Programming Languages](#).

Exercícios

1. Para ser entregue: [Lista 22](#). Lembre-se: respostas escritas à mão.
2. (P.O.F.): Acorde cedo para assistir ao nascer do sol.

[Notas e exemplos](#) usados em aula.

Predicados numéricos em Prolog

Conceitos que devem ser entendidos.

- Prolog possui vários testes de igualdade, por exemplo:
 - **is** A cláusula $x \text{ is } Y$ avalia Y e unifica o resultado com X . A cláusula $3 \text{ is } 1+2$ é verdadeira, mas a cláusula $1+2 \text{ is } 3$ não é. A cláusula $x \text{ is } 1+Y$ produz uma exceção se a variável Y não estiver associada a nenhum valor.
 - **=** A cláusula $x = Y$ unifica X e Y . Unificação não causa a avaliação de valores numéricos. Assim, $3 = 1+2$ não é verdade. Tampouco é verdade $1+2 = 3$.
 - **==** A cláusula $x == Y$ avalia tanto X quanto Y e sucede caso estes termos tenham o mesmo valor. Tanto $3 == 1+2$ quanto $1+2 == 3$ são verdade.
- Prolog é uma linguagem dinamicamente tipada.
- Prolog é uma boa linguagem para formular soluções para problemas NP-completos.
 - Veja, por exemplo, a cláusula **findall**.

Questões para discutir

- A linguagem Prolog deveria ser parte da ementa de linguagens de programação?

Leitura

- [Existem empregos que requerem Prolog?](#)
- [99 Problemas em Prolog](#).
- [Aplicações escritas em prolog](#) (Artigo: Logic Programming Applications: What Are the Abstractions and Implementations?)
- **Importante:** Capítulo sobre *Math in Prolog* do livro [Introduction to Programming Languages](#).

Exercícios

1. Para ser entregue: [Lista 23](#). Lembre-se: respostas escritas à mão.
O autor do livro que adotamos disponibilizou alguns predicados para ajudar nestes exercícios:
 - A versão simples do problema das [oito-rainhas](#).
 - E a versão [complicada](#).
 - O problema da [mochila](#).
2. (P.O.F.): Leia algum autor que já ganhou o Nobel de literatura.

[Notas e exemplos](#) usados em aula.

Semântica Formal

Conceitos que devem ser entendidos.

- Enquanto a sintaxe define a aparência de uma linguagem de programação, a semântica define seu significado.
- Um jeito de definir a semântica formal de uma linguagem $L1$ é construir um interpretador de $L1$ usando uma linguagem $L2$ de semântica bem definida.
 - Mas como definir a semântica de $L2$?
- Existem notações formais para definir semântica de linguagens de programação. Por exemplo: semântica operacional, semântica axiomática e semântica denotacional.
- A semântica operacional especifica, passo a passo, o que ocorre com o programa enquanto ele é executado.

Questões para discutir

- Até que ponto um programador deve conhecer a semântica da linguagem de programação que ele usa?
- Como ter certeza de que o programa faz o que se supõem que ele faça?

Leitura

- [Semântica operacional. Seções 8.5 \(não é preciso ler sobre "completeness and consistency"\) e 8.6.](#)
- **Importante:** Capítulo sobre *Semantics* do livro [Introduction to Programming Languages](#).

Exercícios

1. Para ser entregue: [Lista 24](#). Lembre-se: respostas escritas à mão.
O autor do livro que adotamos disponibilizou os interpretadores em sua página:
 - [Interpretador](#) com escopo dinâmico.
 - [Interpretador](#) com escopo estático.
2. (P.O.F.): Dê uma sugestão para algum professor do departamento de como ele pode melhorar suas aulas.

[Notas e exemplos](#) usados em aula.

História das Linguagens de Programação

Conceitos que devem ser entendidos.

- Algoritmos surgiram muito antes dos transístores.
- Entre as primeiras linguagens de programação, cita-se Plankalkul, Fortran, Lisp e Algol.
- Smalltalk popularizou orientação por objetos.
- SML, Prolog e Java vieram depois.

Questões para discutir

- Pela lei de Moore, a velocidade do Hardware dobra a cada 18 meses. Existe alguma lei semelhante para a produtividade dos programadores?
- Para onde estão indo as linguagens de programação?

Leitura

- [A História das Linguagens de Programação](#)
- [A história das linguagens de programação.](#)
- [Um linha do tempo](#) publicada pela editora O'reilly.
- [Mais linha de tempo](#) (Desta vez a original).
- [How to shoot yourself in the foot](#) using any programming language.
- [Beating the Average](#), um artigo bem legal escrito por Paul Graham.

Lista de Exercícios

1. Para ser entregue: [Lista 25](#). Lembre-se: respostas escritas à mão.
2. (P.O.F.): Escreva uma bela carta e deixe-a para algum sortudo, dentro de um livro da biblioteca.

[Notas e exemplos](#) usados em aula.

Aula de Revisão.

Dicas para se dar bem nesta próxima prova:

1. Entenda as notas e exemplos dados em aula.
2. Leia os [Slides](#) do livro.
3. Faça as listas de exercícios.
4. Não deixe nenhum P.O.F. sem fazer.

Abaixo estão alguns exemplos de provas dados em anos anteriores. A matéria agora é muito maior, afinal, conceitos da primeira metade do curso podem ser cobrados novamente. É claro que a ênfase será sobre a matéria da segunda metade. As dicas são as mesmas da primeira prova, porém desta vez [Build Me Up Buttercup](#) pode não ser suficiente. Assim, se ficarem cansados enquanto treinam fazendo as provas, eu recomendo agora que vocês parem tudo para escutar [Dancing in the Moonlight](#). O solo de piano logo no início da música coloca qualquer um para cima.

- [Prova 1.](#)
- [Prova 2.](#)
- [Prova 3.](#)
- [Prova 4.](#)
- [Prova 5.](#)
- [Prova 6.](#)
- [Prova 7.](#)
- [Prova 8.](#)
- [Prova 9.](#)
- [Prova 10.](#)
- [Prova 11.](#)
- [Prova 12.](#)
- [Prova 13.](#)
- [Prova 14.](#)
- [Prova 15.](#)
- [Prova 16.](#)
- [Prova Online 1.](#)
- [Prova Online 2.](#)
- [Prova Online 3.](#)
- [Prova Online 4.](#)
- [Prova 19.](#)
- [Prova 20.](#)
- [Prova 21.](#)
- [Prova 22.](#)
- [Prova 23.](#)
- [Prova 24.](#)
- [Prova 25.](#)
- [Prova 26.](#)
- [Prova 27.](#)
- [Prova 28.](#)
- [Prova 29.](#)

Seguem abaixo alguns videos sobre o material coberto nesta segunda metade do curso:

- [Aulas 14 e 15.](#)
- [Aulas 16 e 17.](#)
- [Aulas 18 e 19.](#)
- [Aulas 20 e 21.](#)
- [Aulas 22 e 23.](#)

- [Aula 24.](#)
-

Segunda avaliação.

Duração: *90 minutos.*

Valor: *40 Pontos.*

Conteúdo: *Todo o curso.*

Modo: *Sem consulta.*

Exame Especial

Bom, se você ficou de exame especial, então sua situação não é muito melhor que [aquela](#) dos cartagineses frente às legiões de Cipião Emiliano... Mas nem tudo está perdido! Abaixo estão alguns exemplos de provas dos últimos semestres que podem servir como combustível para o heroísmo. Contudo, se lhe faltar o ânimo, eu sugiro escutar um [tango argentino](#) que seja dramático o suficiente.

- [Prova 1.](#)
- [Prova 2.](#)
- [Prova 3.](#)
- [Prova 4.](#)